

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- ... was einen Baum, einen aufspannenden Baum und einen minimalen aufspannenden Baum auszeichnet.
- ... worin sich die Algorithmen von Prim und Kruskal in ihrer Vorgehensweise unterscheiden..

Sie können...

- ... für einen gegebenen gewichteten Graphen schrittweise einen minimalen aufspannenden Baum konstruieren.
- ... den Algorithmus von Prim und den Algorithmus von Kruskal auf Beispiele anwenden.
- ... die Ergebnisse und die Laufzeit der beiden Verfahren grundlegend bewerten.

Graphen vs. Bäume

Graphen vs. Bäume

Definition (ungerichteter) Baum

Ein **(ungerichteter) Baum** ist ein spezieller Typ von Graph $G = (V, E)$, der

- zusammenhängend ist
- keine Zyklen enthält
- und keine gerichteten Kanten besitzt

Damit ergibt sich zwangsläufig, dass es von jedem Knoten eines Baums genau einen Pfad zu jedem anderen Knoten gibt.

Falls G nicht leer gilt: $|V| = |E| + 1$

Graphen vs. Bäume

Wurzel eines (ungerichteten) Baums

Man kann in einem ungerichteten Baum $G = (V, E)$ jeden Knoten $v \in V$ zur Wurzel machen.

Beweis durch ein Gedankenexperiment

Man greife den Knoten v und schüttele den gesamten Graphen. Die übrigen Knoten werden von der Schwerkraft nach unten gezogen und bilden den Baum mit der Wurzel v .

Hinweis

Bei einem gerichteten Baum, bei dem die Kanten eine Richtung haben, ist dies nicht möglich. Zur eindeutigen Beschreibung einer Hierarchie ist also ein gerichteter Baum notwendig. Die bisher betrachteten Bäume waren somit gerichtete Bäume.

Graphen vs. Bäume

Aufspannende Bäume

Gegeben ist ein ungerichteter zusammenhängender Graph $G = (V, E)$, der auch Zyklen enthalten kann.

Ein Graph $G_B = (V, E_B)$ mit $E_B \subseteq E$, der die Eigenschaften eines ungerichteten Baums besitzt (insbes. keine Zyklen), heißt **aufspannender Baum** für den Graphen G .

Hinweis

Der aufspannende Baum muss alle Knoten enthalten, und seine Kanten sind auch im Ursprungsgraph enthalten.

Graphen vs. Bäume

Aufspannende Bäume

Ein Graph kann mehrere aufspannende Bäume besitzen.

Beispiel

Ein vollständiger Graph mit 4 Knoten besitzt 16 aufspannende Bäume mit unterschiedlichen Kantenkombinationen.

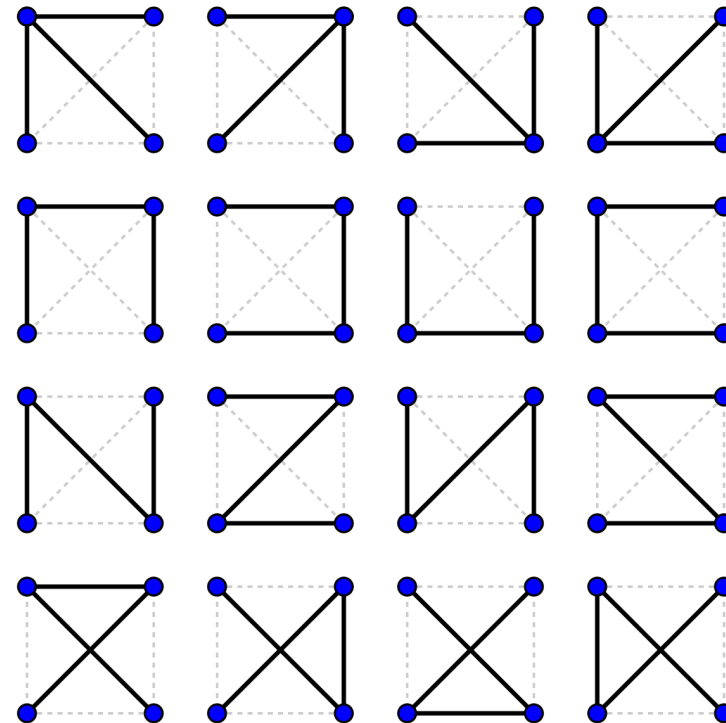


Abbildung: Wikimedia

Minimaler aufspannender Baum (engl.: minimal spanning tree, MST)

Minimaler aufspannender Baum

Definition

Gegeben ist ein bewerteter ungerichteter zusammenhängender Graph $G = (V, E)$, der auch Zyklen enthalten kann.

Ein Graph $G_B = (V, E_B)$ mit $E_B \subseteq E$, der die Eigenschaften eines ungerichteten Baums besitzt (insbes. keine Zyklen), heißt **minimaler aufspannender Baum** für den Graphen G , wenn gilt:

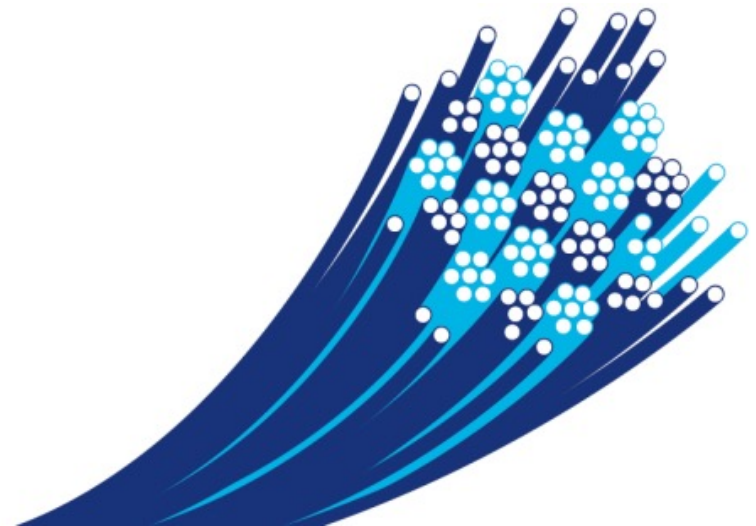
$$\sum_{(u,v) \in E_B} w(u, v) \text{ minimal}$$

Minimaler aufspannender Baum

Anwendungsbeispiel

Bei Infrastrukturmaßnahmen stellt sich gelegentlich die Frage, welche Strecken ausgebaut werden sollen, um eine flächendeckende Versorgung zu gewährleisten.

So könnten z.B. beim Ausbau des Glasfasernetzes Entscheidungen notwendig sein, zwischen welchen Städten/Ortschaften Glasfaserstrecken gebaut werden sollen, so dass anschließend alle Ortschaften mit minimalen Kosten angebunden sind.



Minimaler aufspannender Baum

Greedy-Ansatz

Sei $G = (V, E)$ ein gewichteter ungerichteter zusammenhängender Graph.
Gesucht ist die Menge der Kanten des minimal aufspannenden Baums $E_B \subseteq E$.

„Greedy-Ansatz“

Man startet mit einer leeren Menge A und fügt iterativ immer eine Kante hinzu, die **sicher** zum minimal aufspannenden Baum gehört. Nach $|V| - 1$ Schritten terminiert der Algorithmus.

Ein solcher Algorithmus heißt **greedy** (gierig), weil er sukzessive die Gesamtlösung durch Auswahl einer optimalen Alternative konstruiert, ohne frühere Entscheidungen revidieren zu müssen. Die Herausforderung dabei ist, stets eine **sichere Kante** zu ermitteln.

Algorithmus von Prim

Algorithmus von Prim

Grundidee

Der MST wird iterativ mit einem Greedy-Ansatz gefunden.

Man unterscheidet bei jedem Iterationsschritt zwischen Knoten P , die von den bereits identifizierten Kanten A des MST erreicht werden, und den Knoten $K \subseteq V$, die noch nicht erreicht werden.

Nun wählt man die Kante e mit dem geringsten Gewicht aus, die einen Knoten aus P mit einem Knoten K verbindet und fügt e zur Menge A hinzu.

Der sich nach $|V| - 1$ Schritten ergebende Baum $G_B = (V, A)$ ist der MST von $G = (V, E)$

Algorithmus von Prim

Korrektheitsüberlegung

Ist e eine sichere Kante?

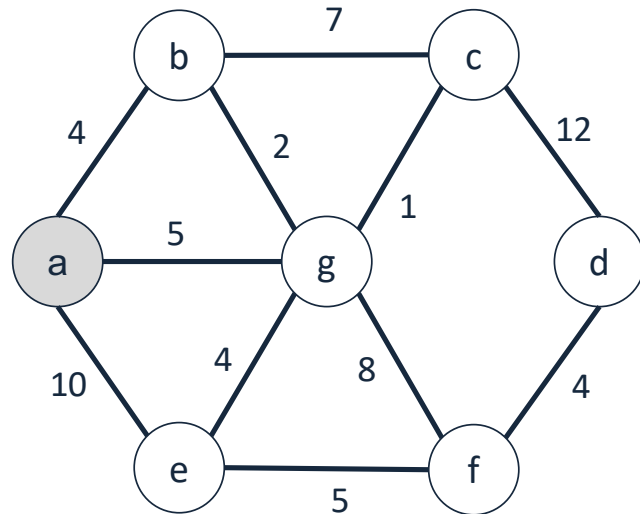
Annahme: Es gibt einen abweichenden MST G'_B , der $e = (u, v)$ nicht enthält.

In diesem MST muss es einen Pfad geben, der die Knoten u und v verbindet. Da e die Knotenteilmengen K und P verbindet, verbindet auch dieser Pfad die Teilmengen, d.h. im Pfad gibt es eine Kante f , die einen Knoten aus K mit einem Knoten aus P verbindet.

Es gilt $w(f) \geq w(e)$, sonst wäre f anstelle von e gewählt worden.

Entfernt man f aus G'_B und fügt stattdessen e ein, so erhält man wieder einen MST. Somit ist e eine sichere Kante.

Algorithmus von Prim

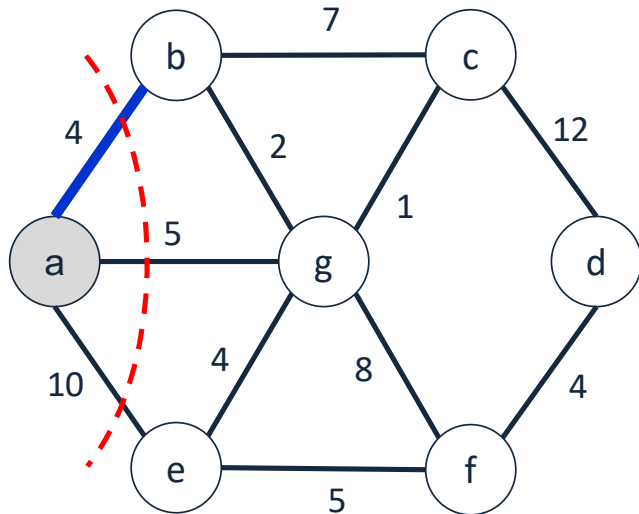


Start

Zu Beginn ist die Kantenmenge A leer.

Da am Ende alle Knoten verbunden sein müssen, kann mit einem beliebigen Knoten in der Menge P begonnen werden (hier: a). Alle anderen Knoten sind in der Menge K .

Algorithmus von Prim

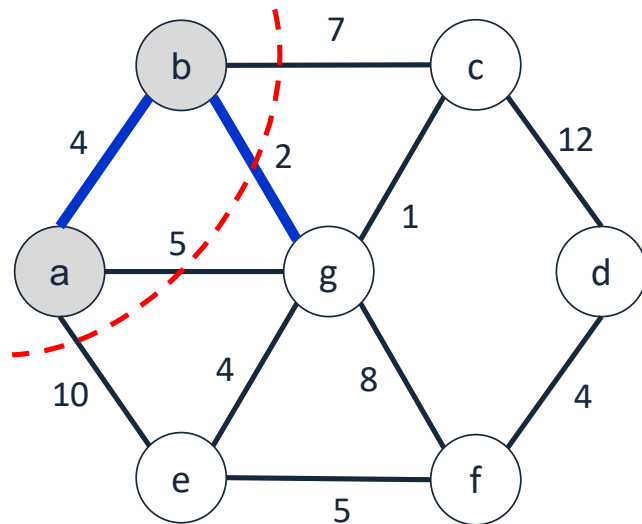


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (a, b) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

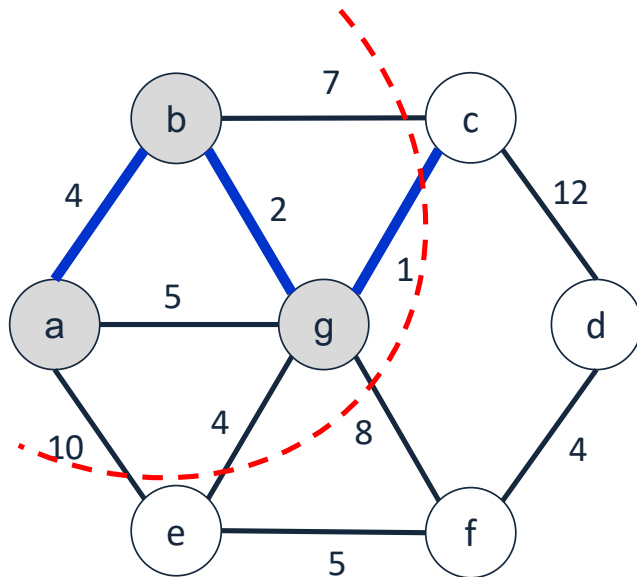


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (b, g) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

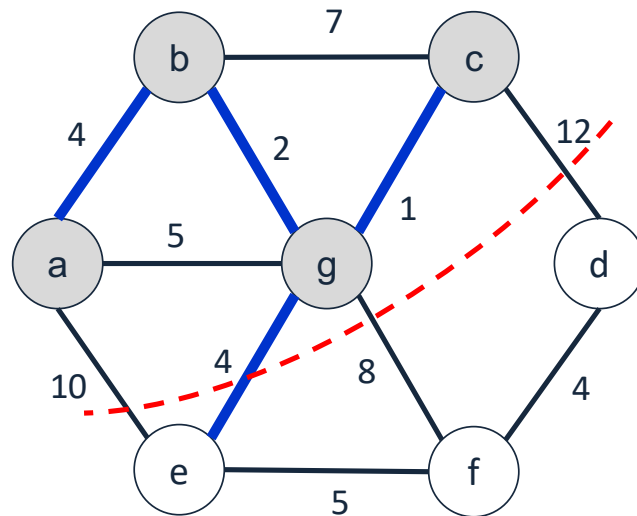


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (g, c) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

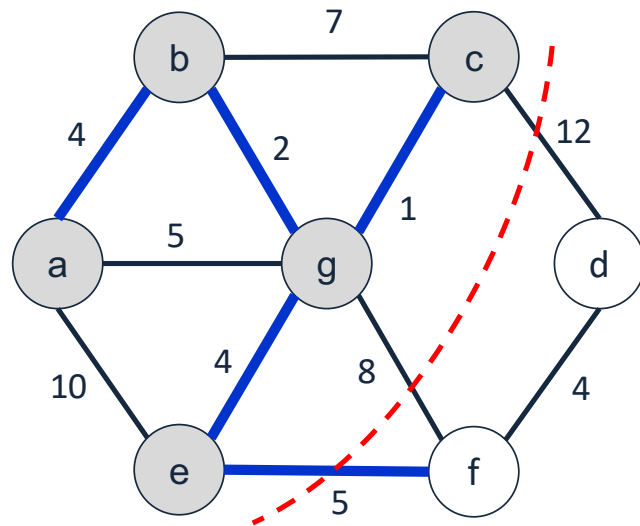


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (g, e) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim

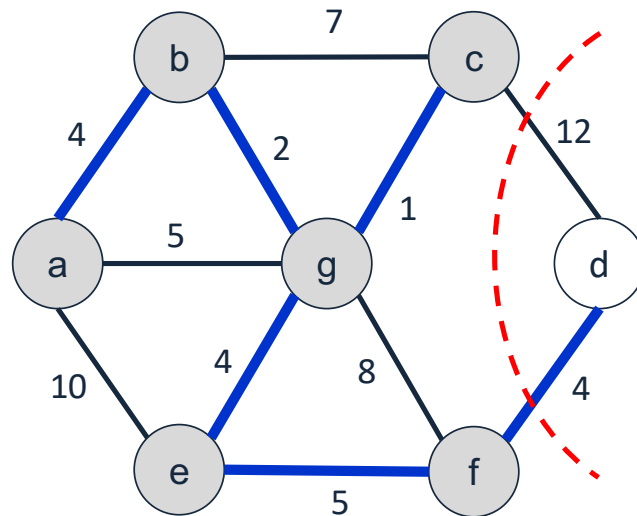


Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (e, f) .

Diese Kante wird in A aufgenommen.

Algorithmus von Prim



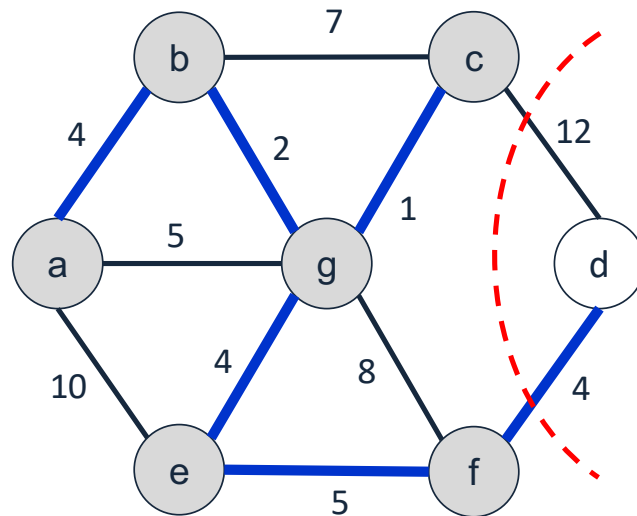
Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (f, d) .

Diese Kante wird in A aufgenommen.

Nach diesen $|V| - 1$ Schritten terminiert der Algorithmus.

Algorithmus von Prim



Auswahl einer Kante

Die Kante mit minimalem Gewicht, die die Knotenmengen P und K verbindet, ist (f, d) .

Diese Kante wird in A aufgenommen.

Nach diesen $|V| - 1$ Schritten terminiert der Algorithmus.

Algorithmus von Prim

Pseudocode

Die Menge $P = V - K$ wird nicht explizit geführt, nur K .

Ausgewählt wird der Knoten v , der aus P mit minimalen Kosten erreicht werden kann.

Anschließend werden diese Kosten bei den in K verbleibenden Knoten aktualisiert.

Am Ende besitzen alle Knoten (bis auf die Wurzel) ihren Elternknoten im MST, woraus sich die Menge A ergibt.

MST-PRIM(V, E, s)

```

1: queue  $\leftarrow$  new PriorityQueue           ▷ Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$                      ▷ Abstand des Knotens  $v$  von  $P$ 
4:    $v.parent \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$                            ▷ Startknoten  $s$  ist in  $P$ 
8: queue.update( $s$ )
9: while queue not empty do
10:   $v \leftarrow queue.extract\_min()$            ▷ Basierend auf  $v.dist$ 
11:  for each  $e : v \rightarrow u \in E$  do         ▷ Alle von  $v$  ausgehenden Kanten
12:    if  $u \in queue \wedge u.dist > w(e)$  then
13:       $u.dist \leftarrow w(e)$ 
14:       $u.parent \leftarrow v$ 
15:      queue.update( $u$ )
16:    end if
17:  end for
18: end while

```

Algorithmus von Prim

Analyse

Die **while**-Schleife wird $|V|$ -mal durchlaufen.

Bei jedem Durchlauf muss der minimal entfernte Knoten ermittelt werden, was mit einem Min-Heap in $O(\log|V|)$ gelingen kann.

Somit ergibt sich eine Komplexität von $O(|V| \cdot \log|V|)$

Jede Kante wird anschließend einmal betrachtet, somit insgesamt

$$O(|V| \cdot \log|V| + |E|)$$

MST-PRIM(V, E, s)

```

1: queue  $\leftarrow$  new PriorityQueue           ▷ Vorrangwarteschlange
2: for each  $v \in V$  do
3:    $v.dist \leftarrow \infty$                  ▷ Abstand des Knotens  $v$  von  $P$ 
4:    $v.parent \leftarrow \mathbf{NIL}$ 
5:   queue.insert( $v$ )
6: end for
7:  $s.dist \leftarrow 0$                        ▷ Startknoten  $s$  ist in  $P$ 
8: queue.update( $s$ )
9: while queue not empty do
10:   $v \leftarrow queue.extract\_min()$         ▷ Basierend auf  $v.dist$ 
11:  for each  $e : v \rightarrow u \in E$  do      ▷ Alle von  $v$  ausgehenden Kanten
12:    if  $u \in queue \wedge u.dist > w(e)$  then
13:       $u.dist \leftarrow w(e)$ 
14:       $u.parent \leftarrow v$ 
15:      queue.update( $u$ )
16:    end if
17:  end for
18: end while

```

Union - Find

Union - Find

Zielsetzung

Für den folgenden Algorithmus benötigen wir eine Möglichkeit,

1. viele einzelne Objekte zunächst als einelementige Mengen zu verstehen (Operation: MAKE-SET),
2. anschließend die Mengen zu vereinigen (Operation: UNION) und
3. schließlich feststellen zu können, ob zwei Objekte Elemente der gleichen Menge sind.

Für die letzte Anforderung definieren wir je Menge ein kanonisches Element (Musterelement), das wir von jedem Element der Menge aus ermitteln können (Operation: FIND-SET). Zwei Objekte sind dann in der gleichen Menge, wenn sie dasselbe kanonische Element besitzen.

Union - Find

Pseudocode

MAKE-SET(v)

1: $\text{parent}[v] \leftarrow v$

FIND-SET(v)

1: **if** $\text{parent}[v] = v$ **then**
2: **return** v
3: **else**
4: **return** **FIND-SET**($\text{parent}[v]$)
5: **end if**

UNION(u, v)

1: $r_u \leftarrow \text{FIND-SET}(u)$
2: $r_v \leftarrow \text{FIND-SET}(v)$
3: $\text{parent}[r_v] \leftarrow r_u$

Algorithmus von Kruskal

Algorithmus von Kruskal

Grundidee

Der MST wird iterativ mit einem Greedy-Ansatz gefunden.

Man beginnt wieder mit $A = \emptyset$ und erweitert A sukzessive durch Auswahl der Kante $e \in (E - A)$ mit dem geringsten Gewicht, die mit den Kanten aus A keinen Zyklus bildet.

Temporär können demnach getrennte Teilbäume entstehen. Am Ende ergibt sich jedoch ein zusammenhängender Baum, weil der Ausgangsgraph zusammenhängend war, also eine verbindende Kante zwischen Teilbäumen existiert, deren Wahl zu keinem Zyklus führt.

Algorithmus von Kruskal

Korrektheitsüberlegung

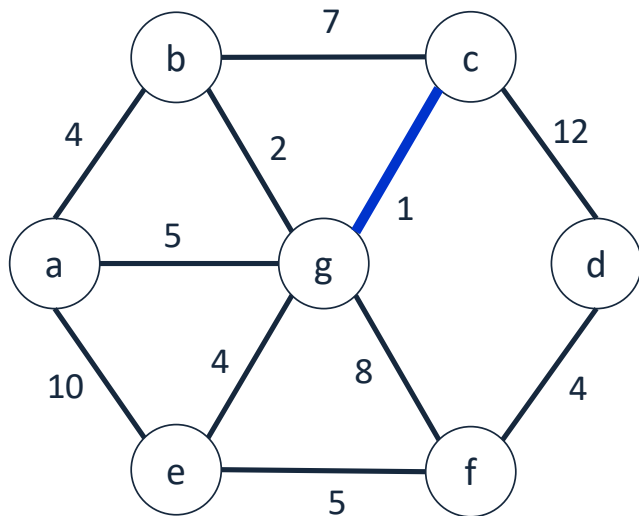
Ist ein so konstruierter $G = (V, A)$ wirklich ein MST?

Beweis durch Induktion, dass während der Konstruktion stets gilt: $A \subseteq E_B$ wobei $G_B = (V, E_B)$ minimaler aufspannender Baum.

Für $A = \emptyset$ ist die Aussage trivial.

Sei e eine nach Vorschrift gewählte Kante. Dann ist auch $A \cup \{e\} \subseteq E_B$, denn andernfalls besäße $E_B \cup \{e\}$ einen Zyklus mit einer Kante f . Entfernt man f aus G_B und fügt stattdessen e ein, so erhält man wieder einen MST. Somit ist e eine sichere Kante.

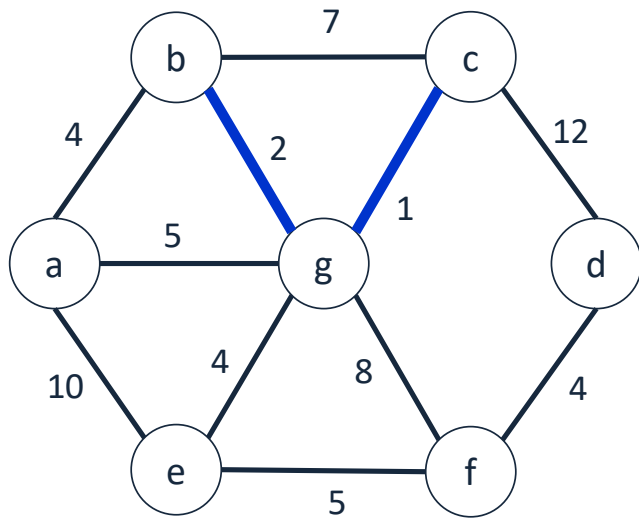
Algorithmus von Kruskal



Start

Im ersten Schritt wird die Kante mit dem geringsten Gewicht ausgewählt und zu A hinzugefügt.

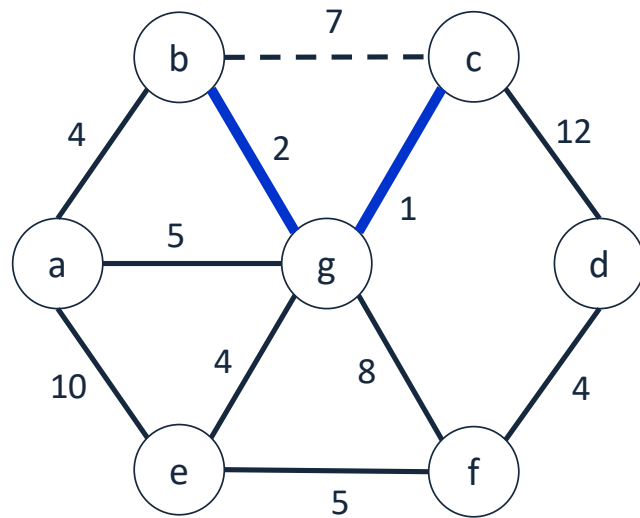
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

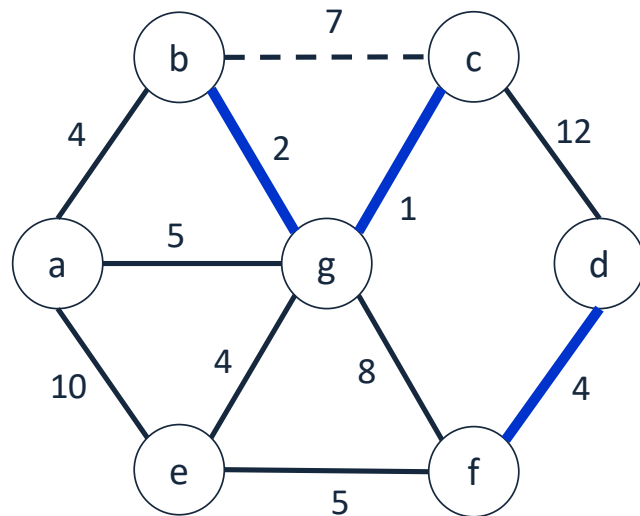
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

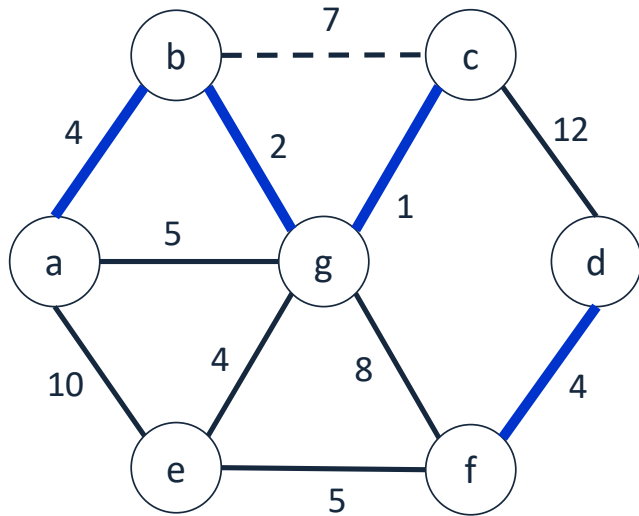
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

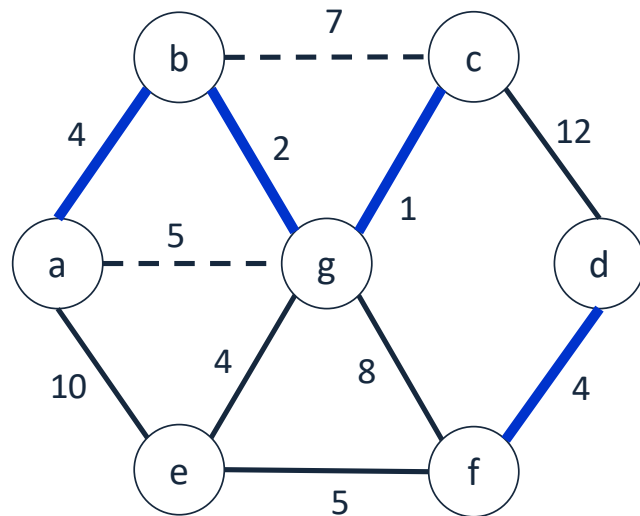
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

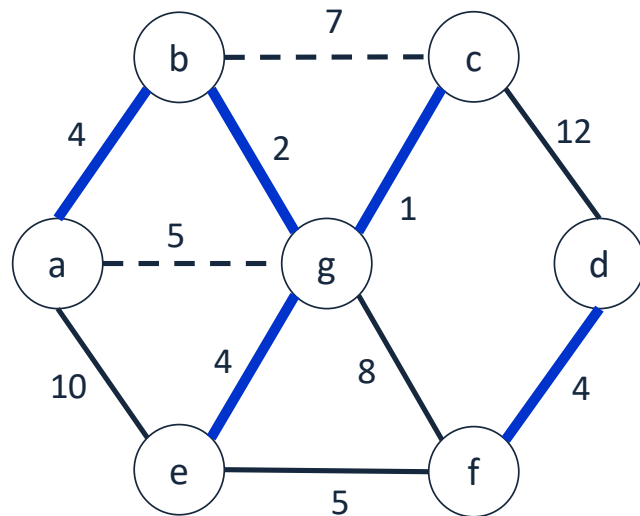
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

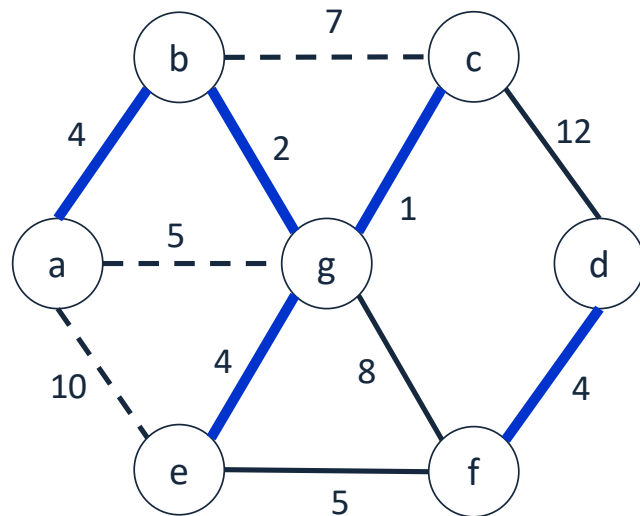
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

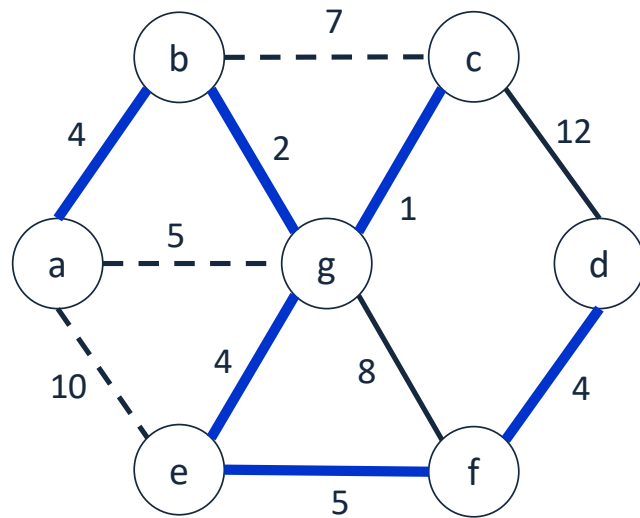
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

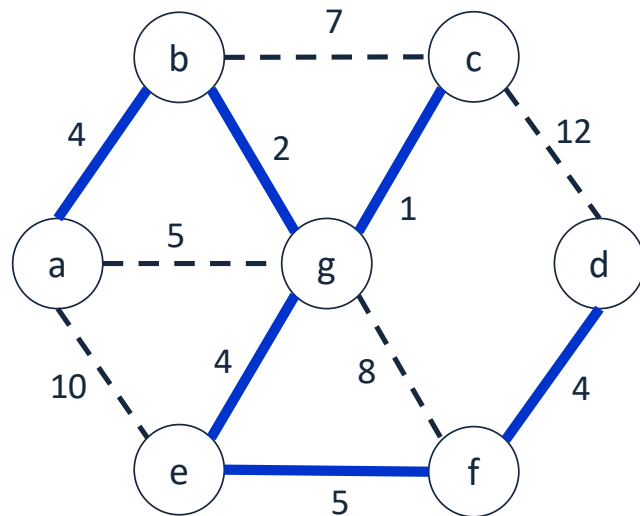
Algorithmus von Kruskal



Finden der nächsten Kante

Die verbleibenden Kanten werden aufsteigend nach Kantengewicht untersucht. Sofern kein Zyklus entsteht, wird die Kante in A aufgenommen.

Algorithmus von Kruskal



Finden der nächsten Kante

Kann keine weitere Kante aufgenommen werden, ohne einen Zyklus zu erzeugen, terminiert der Algorithmus.

Algorithmus von Kruskal

Pseudocode

Die Laufzeit wird dominiert durch das Sortieren in Zeile 5, das $O(|E| \log|E|)$ benötigt.

Bei effizienter Implementierung der Mengenoperationen erhält man insgesamt $O(|E| \log|V|)$

MST-KRUSKAL(V, E)

```

1:  $A \leftarrow \emptyset$ 
2: for each  $v \in V$  do
3:   MAKE-SET( $v$ )
4: end for
5: SORT( $E$ )
6: for each  $e : v \rightarrow u \in E$  do
7:   if FIND-SET( $v$ )  $\neq$  FIND-SET( $u$ ) then
8:      $A \leftarrow A \cup \{e\}$ 
9:     UNION( $v, u$ )
10:  end if
11: end for

```

▷ Basierend auf w
 ▷ Alle Kanten

Vielen Dank für die Aufmerksamkeit!